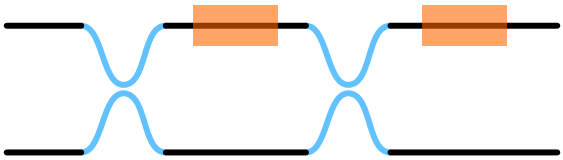
Optical Machine Learning Project: MOCS-3D

MANTIS



Project Report

Tim Gühring • Leah Haase

August 2, 2026

Friedrich Schiller University Jena

Source code and data available on

GitHub

Contents

1 Introduction & System overview **3**

2 System implementation **4**
2.1 Loading and preprocessing MNIST data 4
2.2 Linear Regression 4
2.3 Optical Machine Learning 5
2.3.1 Encoding 5
2.3.2 Processing 5
2.3.3 Decoding 8
2.3.4 Gradient-Descent Training Through Adjoint-based Backpropagation . . . 8

3 Training and performance evaluation **10**
3.1 Linear Regression 10
3.2 Initialization of mesh's phase shift parameters and layer number 10
3.3 Learning rate analysis 12
3.4 Side length analysis (down-sampling) 13
3.5 Batch size analysis 13
3.6 Loss function comparison 13
3.7 Hyperparameter theta_enc (in phase encoding) 14
3.8 Our best model 14
3.9 Outlook 14

4 Discussion and critical analysis **15**
4.1 Bias-variance trade-off analysis 15
4.2 Implementation challenges 15
4.3 Estimated energy consumption 15

5 Conclusion **16**

Appendices **19**

A Analytical Derivation of the Local Gradient **19**

B Additional figures **21**
B.1 Learning rate analysis 21
B.2 Side length analysis (down-sampling) 22
B.3 Batch size analysis 22
B.4 Hyperparameter theta_enc (in phase encoding) 23
B.5 Architecture comparison 23

Acronyms

AVM adjoint variable method. 8, 19

ML machine learning. 3

MZI Mach-Zehnder interferometer. 3, 5, 6, 8, 11, 15, 19, 20, 23

NN neural network. 3

OIU optical interference unit. 3, 6

ONU optical nonlinearity unit. 3, 15

1 Introduction & System overview

Machine learning (ML) has emerged as a powerful paradigm for applications that rely on data-driven optimization, improving system performance by iteratively training on data. Following the introduction of the perceptron by Rosenblatt, Rumelhart et al. demonstrated that neural network (NN) with hidden layers, trained via backpropagation, can solve non-linearly separable problems such as the XOR problem [1, 2]. This laid the groundwork for a broad range of applications, ultimately enabling the era of transformer-based [3] and generative AI models [4, 5]. The performance of such deep-learning models was found to scale predictably as a power law with model size, dataset size, and training compute [6, 7]. This scaling comes at a substantial energy cost: In 2025 data centers are estimated to account for 1.5% of global electricity generation, driven increasingly by AI workloads [8, 9].

Photonic hardware offers a promising substrate for running neural networks more efficiently. Cybenko and Hornik et al. showed in 1989 that neural networks can approximate any continuous function, given tunable weighted interconnections and non-linear activation functions [10, 11]. Optical systems can provide both. As early as 1994, Reck et al. demonstrated that any unitary operator can be realized with optical components, enabling arbitrary linear transformations in the optical domain [12]. Among the various optical neural network (ONN) implementations proposed since, on-chip meshes of Mach-Zehnder interferometers (MZIs) stand out for their scalability [13, 14]. Built from single-mode phase shifters and 50:50 beam splitters, such a mesh acts as an optical interference unit (OIU) that carries out the linear part of the computation. A schematic sketch of the interferometer is shown in Figure 1. The remaining optical nonlinearity unit (ONU) can be realized through nonlinear optical materials, saturable absorbers, optoelectronic activation, or, as in our system, the intensity detection at the output itself [14, 15].

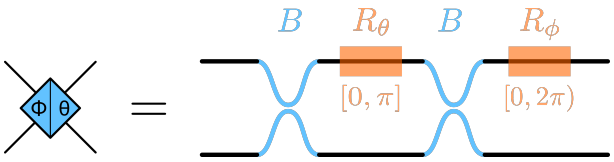


Figure 1: Illustration of a MZI composed of beam splitters B and phase shifters R. The MZI is uniquely characterized by its internal and external phase shift parameters θ and ϕ. The black lines represent single mode waveguides, forming the beam splitter using directional coupling.

Training a NN means adjusting its parameters such that its predictions match the true result as closely as possible. This is formalized through a loss function $\mathcal{L}$, which quantifies the discrepancy between the network output and the desired output for a given input. Training then reduces to a minimization problem: the parameters are iteratively updated along the negative gradient of $\mathcal{L}$ until a minimum is reached. The practical feasibility of this scheme rests on the backpropagation algorithm [2]. By applying the chain rule from the output layer backwards, the derivatives with respect to all parameters are obtained in a single pass. This prevents the cost of a full gradient evaluation from growing with the number of parameters.

The matrix–vector products required for training can be executed on the physical system itself [16]. Propagating an error field backwards through the device then yields the gradient needed for gradient descent. This error field is determined by the deviation of the output from the desired result. The central advantage of adjoint-based backpropagation, used in this work, is its scaling [17]. The cost of a full gradient evaluation is independent of the number of parameters. On hardware, three in-situ intensity measurements after each phase shifter suffice to recover the gradient for all parameters at once. Every parameter can then be updated simultaneously to approach a local minimum of the loss landscape.

The objective of this work is to implement a binary classifier using the MZI mesh as the physical core. The model will be trained with samples of handwritten digits “1” and “7” from

the MNIST dataset, using the adjoint-based backpropagation. After the processing layer, two detectors at arbitrary channels will measure the intensity and make a prediction based on the winner-takes-it-all principle. The complete setup is shown in Figure 2. We expect to achieve a high testing accuracy with both the linear regression and the optical model. In contrast to the linear regression, the optical model has a non-linearity at the detection, which might be an advantage for non-linearly separable problems.

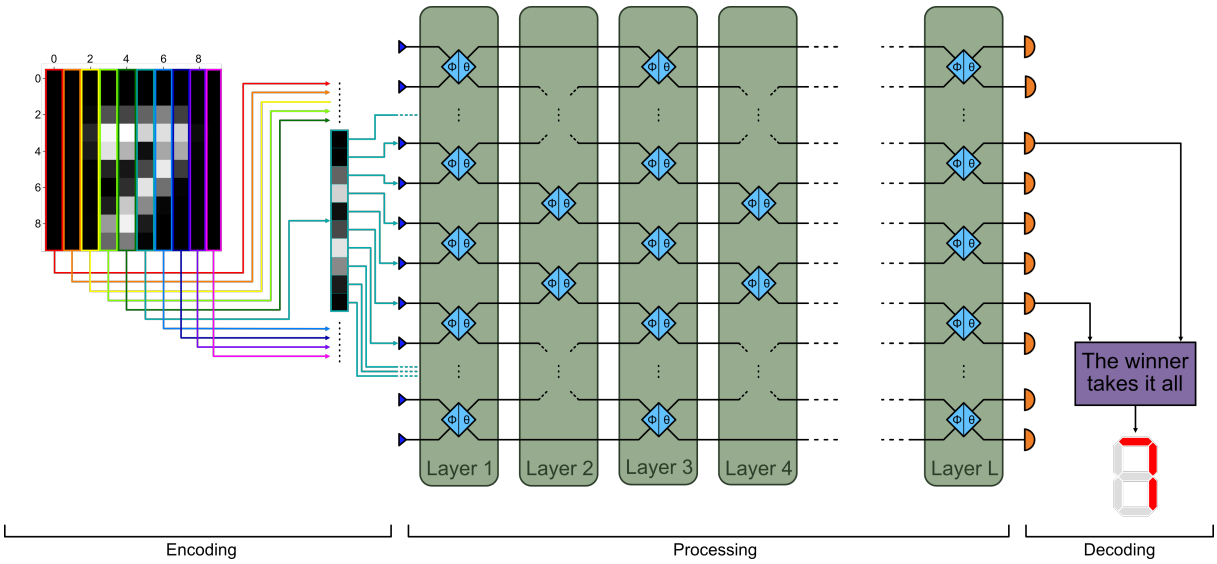


Figure 2: Scheme illustrating the encoding, processing and decoding steps in the optical model for an image of the MNIST dataset.

2 System implementation

2.1 Loading and preprocessing MNIST data

The MNIST dataset is loaded in the function `get_data` in file `encoding.py` using the function `keras.datasets.mnist.load_data`. The images depicting the digits "1" and "7" are selected. Using the `get_data` function, the desired number of samples can be specified and the distribution of both digits in the dataset can be balanced. The split ratio of the dataset in training and testing data can be chosen as well, with a default value of 0.8. The images have a side length of 28 px and are down-sampled using the `skimage.transform.resize` function in the function `down_sample`. The default side length is 10 px. Then, the images are reshaped to an one-dimensional vector and normalized using the function `reshape_and_normalize`. The principle of the reshaping is illustrated in Figure 2. Furthermore, the energy in each image vector can be normalized in the function `get_data` by dividing it by its norm.

2.2 Linear Regression

The linear regression is implemented using the Moore-Penrose pseudoinverse in the function `linear_regression` in `baseline.py` (see Fig. 3). Based on predicted labels `y_pred` and the true labels, the confusion matrix and the accuracy for the binary classification are calculated.

```python
F = np.vstack([np.ones(len(X_train.T)), X_train]).T
theta = np.linalg.pinv(F) @ y_train

y_pred = theta[0] + np.sum(theta[1:] * X_test.T, axis = 1)
y_pred_label = (y_pred >= 0.5).astype(np.float64)
```

Figure 3: Excerpt from function `linear_regression` in `baseline.py`.

2.3 Optical Machine Learning

2.3.1 Encoding

To encode information optically, we use the electromagnetic wave propagating through the waveguide channels of the MZI mesh. Since Maxwell's equations couple the electric and magnetic fields, it is sufficient to describe the wave by one of them, and therefore we work with the electric field throughout. For a monochromatic wave of angular frequency ω propagating along the waveguide axis with wave vector $\vec{k}$, the field in mode j can be written as:

$$E_j(\vec{r}, t) = A_j(\vec{r})\, e^{i(\vec{k}\cdot\vec{r}+\omega t)}.$$

Here we consider only a single scalar component of the field with amplitude A_j, omitting the vector character. This is justified since we assume that all channels carry the same guided mode with a fixed polarisation. For an uniform chip architecture, the channels acquire the exact same phase by propagating along the optical axis. Since the information is carried by the relative differences between the channels, we express the relevant components by

$$\mathbf{E}(\vec{r}, t) = e^{i(\vec{k}\cdot\vec{r}+\omega t)} \begin{pmatrix} A_1(\vec{r})\, e^{i\varphi_1} \\ \vdots \\ A_m(\vec{r})\, e^{i\varphi_m} \end{pmatrix},$$

where φ_j represents the initial phase given to the electric field in mode j. The state of the system is therefore fully described by this vector of complex amplitudes, which is the quantity propagated in our simulation.
The pixel values $V(x, y)$ of an image are imprinted on this field by modulating either the amplitudes A_j or the initial phases φ_j (see Fig. 4). When applying amplitude encoding, the normalized pixel values are mapped onto the field amplitudes while the phase is kept constant, with $E_j = \theta\, V_j$ and $\varphi_j = 0$. Phase encoding is an alternative in which the pixel values are mapped onto φ_j at constant amplitude. The hyperparameter θ is controlling the encoding strength.

```python
def amplitude_encoding(image, theta_enc=1):
    """Encode image information in amplitude of electric field."""
    return (image * theta_enc).astype(complex)

def phase_encoding(image, theta_enc=1):
    """Encode pixel values in the phase, constant amplitude."""
    return np.exp(1j * theta_enc * np.pi * image)
```

Figure 4: Excerpt from `encoding.py` showing the functions `amplitude_encoding` and `phase_encoding`.

2.3.2 Processing

The processing is done by the MZIs in the mesh, that can be set to perform any unitary transformation on the electric fields in each channel [12]. Consisting of an arrangement of beam

splitters B and phase shifters R (see Figure 1), the MZI implements any arbitrary Givens rotation $T = R_\phi B R_\theta B$:

$$T = \frac{1}{2}\begin{pmatrix} e^{i\phi} & 0 \\ 0 & 1 \end{pmatrix}\begin{pmatrix} 1 & i \\ i & 1 \end{pmatrix}\begin{pmatrix} e^{i\theta} & 0 \\ 0 & 1 \end{pmatrix}\begin{pmatrix} 1 & i \\ i & 1 \end{pmatrix}.$$

Permutating their placements in the mesh along the layers allowing mixing between all channels, builds the OIU that can form any unitary operation. Applying these rotations block-wise to their respective channels, this yields the transfer-matrix of layer L, allowing the full propagation by matrix multiplication.

$$\mathbf{E_{out}} = \hat{W}_L \hat{W}_{L-1} ... \hat{W}_1 \mathbf{E_{in}} = \hat{W} \mathbf{E_{in}}. \tag{1}$$

Note that a layer here refers to the depth of one MZI (2 beam splitters and 2 phase shifters) and is not to be confused with the definition of one layer for the adjoint-based backpropagation. There each layers is representing one tunable parameter (1 beam splitter and 1 phase shifter).

Accordingly, the transfer matrices are calculated using functions `beamsplitter`, `phase_shift` and `U` (see Fig. 5). Additionally, two functions `U_theta` and `U_phi` are defined which allow a separate calculation of the transfer matrices for each combination of a phase shifter and beam splitter separately for the gradient-descent algorithm.

```python
def beamsplitter(eta=1.0):
    """Transfer matrix of a beamsplitter with power transmission eta."""
    B_ideal = (1/np.sqrt(2)) * np.array([[1, 1j],
                                         [1j, 1]], dtype=np.complex128)
    return np.sqrt(eta) * B_ideal

def phase_shift(alpha):
    """Transfer matrix to apply a phase shift to upper mode."""
    return np.array([[np.exp(1j*alpha), 0],
                     [0, 1]], dtype=np.complex128)

def U(theta, phi, eta=1.0):
    """Transfer matrix of the MZI with phase shifts theta and phi.
    eta: Power transmission per beamsplitter. Two beamsplitters per MZI."""
    B = beamsplitter(eta)
    return phase_shift(phi) @ B @ phase_shift(theta) @ B
```

Figure 5: Excerpt from `mesh.py` showing the functions `beamsplitter`, `phase_shift` and `U`.

The class `MZIMesh` is a central part of the processing (see Fig. 6). It allows the definition of the chip structure. The implemented options include a rectangular and a triangular mesh of MZIs. According to Pai et al., convergence can be vastly accelerated by using the correct initialization. The "Haar initialization" introduced in [15], is based on globally approximating a uniformly random unitary matrix rather than just randomizing individual components. Thus, position-dependent MZI split ratios form a global Haar-random matrix, ensuring that input light is evenly distributed across all spatial modes. The code for the generation of the rectangular mesh is shown in Figure 7 as an example. Parameters, like the values of `theta` and `phi` for each phase shifter are stored separately.

MZIMesh

`N` : number of single-mode waveguides, $N = m^2$
`plan` : layer list defining the topology

`n_layers`, `slot_counts`, `n_mzis` *(properties)*

`init_random(rng)` : Uniform random initialization of phase shifts
`init_haar(rng)` : Haar initialization of phase shifts
`sensitivity_index()` : Calculates the structural sensitivity index for each MZI (each components global influence)
`layer_matrices`$(\theta, \phi, \eta_{bs}, \alpha_f)$: Returns a list of transfer matrices for each layer of MZIs
`layer_matrices_separate`$(\theta, \phi, \eta_{bs}, \alpha_f)$: Returns a list of transfer matrices for each layer of phase shifter and beam splitter

Figure 6: Class diagram for class MZIMesh from mesh.py

```python
def plan_rectangular(N, L):
    """Clements / rectangular mesh: L Layers, with alternating offset 0 and 1.
    Universal for L = N. Returns: list of ("mzi", ks)."""
    plan = []
    for l in range(L):
        start = l % 2
        ks = list(range(start, N - 1, 2))
        plan.append(("mzi", ks))
    return plan
```

Figure 7: Excerpt from `mesh.py` showing the function `plan_rectangular`.

The functions which are most relevant for the propagation of the electric field in the chip are summarized in Figure 8. The function `forward` returns the electric field at the output ports of the chip on the right, when an electric field has entered through the input ports on the left, while the function `backward` gives the output field for the reverse propagation direction from the right to the left. The output of the forward propagation is calculated by applying the transfer matrix $\hat{W}_L$ of each layer which are stored in `layers` to the incoming electric field vector. For the backward propagation the transfer matrices are transposed and their order is reversed, then they are applied to the electric field vector incoming from the right.

```python
def forward(E_in, layers):
    """Forward propagation through given layers."""
    for Ml in layers:
        E = Ml @ E
    return E

def backward(lam_out, layers):
    """Backward propagation through reversed layers."""
    return forward(lam_out, adjoint_layers(layers))

def adjoint_layers(layers):
    """Transposing layers and reversal of their order."""
    return [Ml.T for Ml in reversed(layers)]
```

Figure 8: Shortened excerpt from `processing.py` showing the functions `forward`, `backward` and `adjoint_layers` which are relevant for the electric field propagation through the chip.

2.3.3 Decoding

The detected intensity is calculated as the absolute squared value of the electrical field for each output: $I_n = |E_{out,n}|^2 = E_{out,n} \cdot E^*_{out,n}$. Two specific output ports are assigned to digits "1" and "7" respectively. The digit assigned to the port with the higher intensity is returned as the predicted label. This is implemented in the functions `detection` and `_detector_scores` in `decoding.py`.

2.3.4 Gradient-Descent Training Through Adjoint-based Backpropagation

Conventional backpropagation operates on a known computational graph and requires the transpose of each weight matrix to be available as an explicit numerical object. In our system the weights are not numbers stored in memory but the phase settings of the MZIs, and the linear operation they implement is a propagating optical field. What is therefore needed is a formulation of the same gradient in terms of the physical parameters of the device.

The adjoint variable method (AVM) provides exactly this formulation. It originates in the calculus of variations and was later adapted to engineering sensitivity analysis. Today it is a standard tool of photonic inverse design [18–20]. Its key result is that the derivative of a scalar figure of merit with respect to all design parameters follows from only two field solutions. The first is the original field, driven by the physical input source. The second is an adjoint field, driven by a source that encodes the error at the output [17]. The gradient with respect to an individual parameter is then given by the local overlap of these two fields at its position. For the calculation of the gradient, we use the following equation, which is derived in appendix A:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}p} = -2\ \mathrm{Im}\left\{A_p\, A_p^{\mathrm{adj}}\right\} \tag{2}$$

Summarizing this derivation, that is specifically matched to our system, we get the following practical procedure for our training:

1. Propagating the input field $\mathbf{E}_{in}$ (encoded MNIST data) using forward propagation according to equation (1). After every phase shifter (ϕ and θ), the electrical field in that channel is stored in a list.

2. Decoding the output and calculating the loss for the chosen cost function. Determining the adjoint vector for the backpropagation.

3. Propagating the adjoint field using backward propagation through the transposed layer transfer matrices. After every phase shifter (ϕ and θ), the electrical field in that channel is stored in a list.

4. Calculating the derivative of the loss with respect to each phase shifter using equation (2) by interfering the forward and backward propagated fields at each position. The parameters ϕ and θ are updated accordingly using gradient-descent.

The class `trainer` is used for the training of the model, i.e. the optimization of the applied phase shifts. The attributes and functions of the class are shown in the class diagram in Figure 9. The algorithm of the function `fit` of the `trainer` class is depicted as a flowchart in Figure 10. Figure 11 shows the implementation of equation (2) for the calculation of the gradient at each phase shifter in function `gradient` in the code.

Trainer

`mesh`, `cfg` : architecture and hyperparameters
`thetas`, `phis` : trainable phase shifter settings $\theta_{n\ell}, \phi_{n\ell}$
`history` : `loss`, `acc`, `grad_norm` per epoch, `batch_loss` per step
`train_time`, `test_acc` : performance metrics

`_layers()` : Build the transfer matrices for the current weights
`step(E,y)` : Perform one gradient step on one mini-batch
`evaluate(E,y)` : Perform testing
`fit(E,y)` : Perform training
`inference_time(E, repeats)` : Returns time required per single classification
`save(path)` : Store the run as a JSON file
`load(path)` : Rebuild a trainer from a stored JSON file

Figure 9: Class diagram for `trainer` class from `trainer.py`.

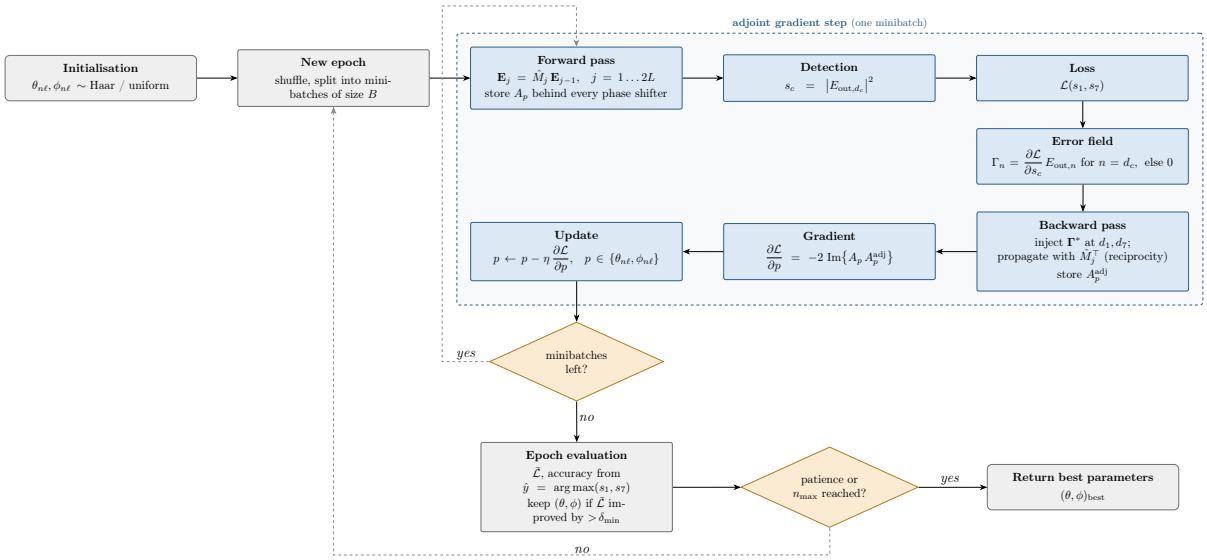


Figure 10: Flowchart of `fit`-algorithm in `trainer` class.

```python
def gradient(fh, bh, plan):
    """Calculation of gradient for all values of theta and phi based on forward and backward
    propagating fields"""
    grad_th, grad_ph = [], []
    for l, (kind, ks) in enumerate(plan):
        gt = np.zeros(len(ks)); gp = np.zeros(len(ks))
        for i, k in enumerate(ks):
            # phase_shift applied to upper mode only
            gt[i] = -2*np.imag(np.sum(bh[2*l+1][k] * fh[2*l+1][k]))
            gp[i] = -2*np.imag(np.sum(bh[2*l+2][k] * fh[2*l+2][k]))
        grad_th.append(gt); grad_ph.append(gp)
    return grad_th, grad_ph
```

Figure 11: Shortened excerpt from `processing.py` showing the function `gradient`.

The loss function $\mathcal{L}(s_1, s_7)$ depends on the scores of the chosen detectors for the digits 1 (s_1) and 7 (s_7). Two loss functions are implemented in `decoding.py`: mean square error (function `mse`) and softmax-cross-entropy (function `softmax_ce`). The mean squared error loss and its derivative with respect to the detector score can be defined as:

$$\mathcal{L}_{\text{MSE}}(s_1, s_7) = \sum_{c\in\{1,7\}} (s_c - t_c)^2 \text{ and } \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial s_c} = 2(s_c - t_c).$$

The softmax-cross-entropy loss and its derivative are given by:

$$\mathcal{L}_{\text{CE}}(s_1, s_7) = -\sum_{c\in\{1,7\}} t_c \log(p_c) \text{ and } \frac{\partial \mathcal{L}_{\text{CE}}}{\partial s_c} = p_c - t_c \text{ with } p_c = \frac{e^{s_c}}{e^{s_1} + e^{s_7}},$$

where t is the target value.

3 Training and performance evaluation

Over the whole project we stuck to the dataset of 1000 images of each class, split into 80% training, 20% testing using the function `get_data` with the randomness seed "1550", our results are reproducible. The used parameters and results of the given tests can be retrieved from the JSON files in the results folder.

3.1 Linear Regression

As a baseline to compare our optical machine learning model against, a linear regression is performed. The results of the linear regression with and without energy normalization and for different values of the image side-lengths ($m = [1, 2, 4, 6, 8, 10, 16, 20, 28]$) are compared. A side-length of $m = 6\,\text{px}$ with energy normalization yields the highest testing accuracy with a value of 0.998. For all m-values except $m = 1$ and $m = 2$, the energy normalization method yields better results. The superior performance of the non-normalized case for these low m parameters is explained by the fact, that all of the information of the picture is contained in only one and 4 pixel respectively. In this case, the model is basically trained on the "used ink" to draw the digit. This is also the reason why the energy-normalized mode performs better for larger m. If more information-carrying features are available, the information about overall energy in the system is more prone to generate confusion, for example due to a thicker written "1". The high accuracy of the linear regression shows, that the given classifying problem is almost linearly separable.

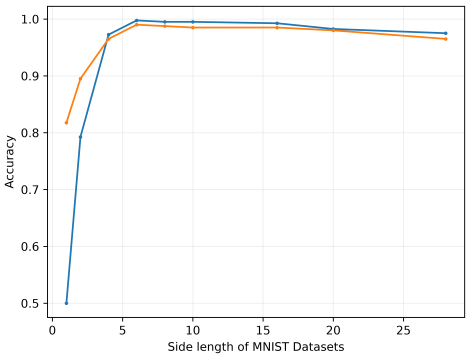


Figure 12: Accuracy of the linear regression with (blue graph) and without (orange graph) energy normalization for different side lengths of the images.

3.2 Initialization of mesh's phase shift parameters and layer number

The behavior and performance of our model is examined for the two different initialization methods. In Figure 13, the intensities of an electric field at input port 50 are visualized. It is visible that for the Haar initialization, the electric field reaches the edges of the mesh much faster than for the random initialization.

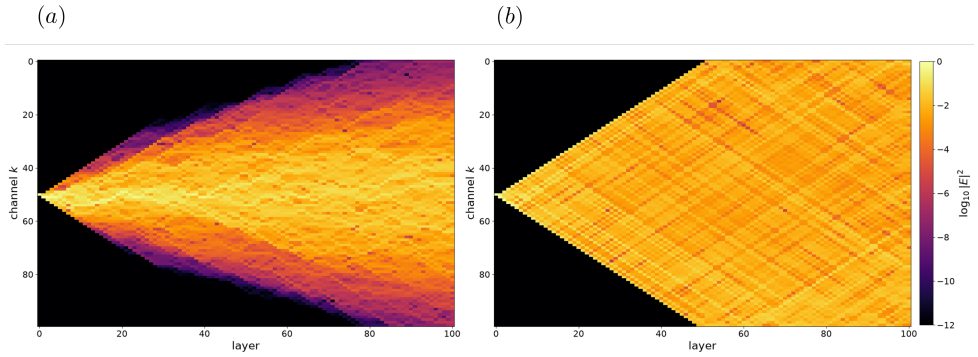


Figure 13: Intensity plot of a normalized electric field starting from channel 50. The mesh's parameters are initialized according to (a) uniform initialization, (b) Haar initialization for uniformly random global transfer matrices.

The training and test results are shown in Figure 14. The training loss of the Haar initialization decreases faster than for the random initialization, leading to a faster convergence. The random initialization reaches a training accuracy of 0.958 after 29 epochs or 355 s. The testing accuracy is 0.980. The Haar initialization reaches a training accuracy of 0.958 after 16 epochs or 185 s, while the testing accuracy is 0.970.

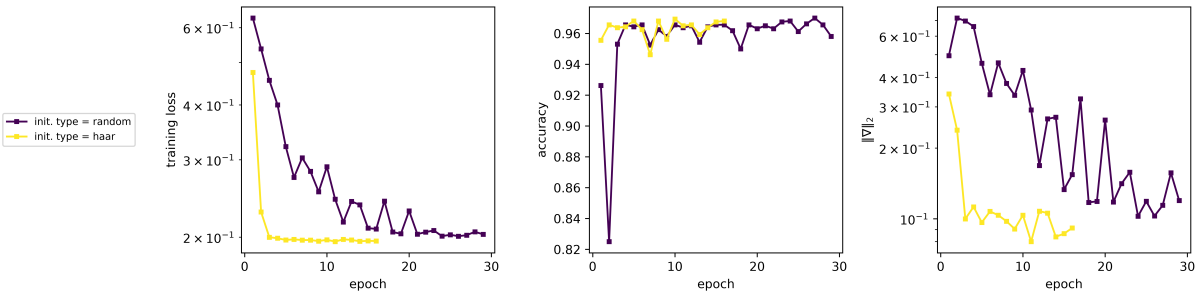


Figure 14: Training loss, training accuracy and normalized gradient for random and Haar initialization.

To see impact of different MZI meshes with respect to the layer number, we perform training over different values. Since the haar initialization promises for faster mixing of the signal inside the mesh over random initialization, we perform this training for both. For Haar initialization, the performance of our model for different numbers of layers in the MZI mesh is shown in Figure 15. The training loss decreases most for the layer counts of 100 and 150, and least for the lowest layer count of 10. The training loss for 200 layers behaves similar to the next lower examined layer counts, but is slightly higher. The training accuracy is highest for the layer counts of 100, 150 and 200. For the random initialization, see Figure 16, the relative performance is equivalent with higher layer counts converging faster and yielding higher accuracies. Compared to the previous initialization however, the slower convergence becomes clearly visible for all layer sizes. Also the maximum achieved testing accuracy with 0.968 is subordinate to the value of 0.975 of the Haar initialization. Since less layers mean a less complex physical setup and consequently less losses, the Haar initialization allows a smaller setup.

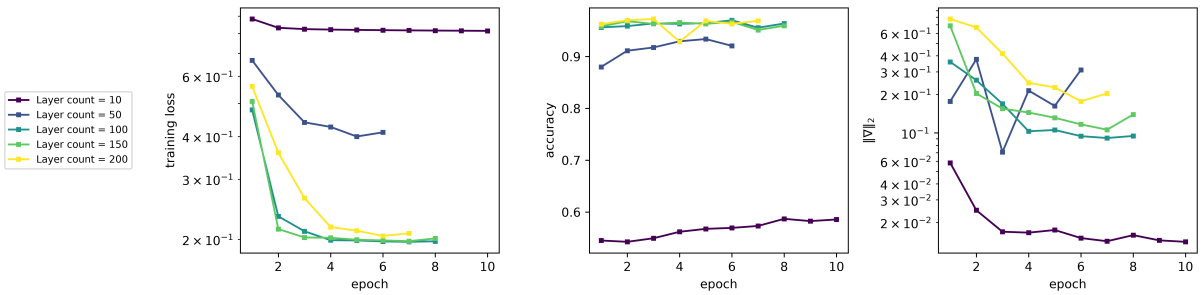


Figure 15: Training loss, training accuracy and normalized gradient for different numbers of layers with Haar initialization.

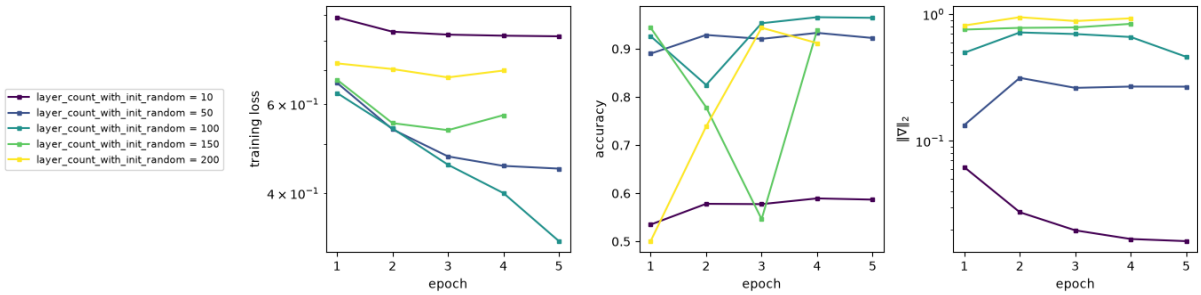


Figure 16: Training loss, training accuracy and normalized gradient for different numbers of layers with random initialization.

3.3 Learning rate analysis

Sweeping over different learning rates is performed to see the effect on the convergence speed and the achieved accuracy. We limit the runtime by using a patience value of 5 and a `min_delta=1e-4`. Therefore, we make sure not to waste training time when no further improvement or no convergence at all is possible. We run it over three different batch sizes (256, 64 and 4). We begin by using 64 to find the best performing learning rate (see Fig. 17) and use 256 and 4 (see appendix B.1) to see how this optimal learning rate differs based on batch sizes.

The highest learning rates converge fast but reach only a low training accuracy. It is clearly visible how lower learning rates lead to a slower but more stable convergence. This behavior is confirmed for all three tested batch sizes. Based on the results, a reasonable trade-off seems to be reached for learning rates of 1.0 to 1.7.

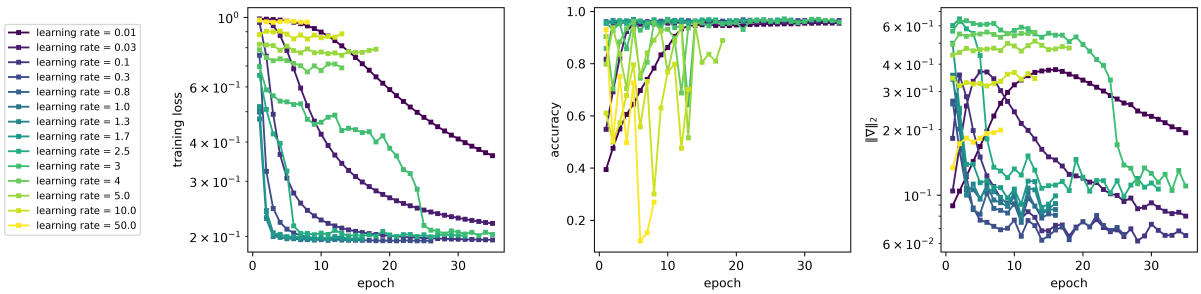


Figure 17: Training loss, training accuracy and normalized gradient for different learning rates for a batch size of 64.

3.4 Side length analysis (down-sampling)

The performance for different side lengths of the down-sampled MNIST images was investigated for side lengths of $m = [6, 8, 10, 14, 18, 22]$. The detector positions are scaled with the side length and placed at one third and two thirds of the total number of output ports. The results are shown in Table 1 (see also Figs. in appendix B.2). The achieved training loss, training accuracy and normalized gradient are similar over all tested side lengths, except for the lowest side length, which shows the worst training accuracy. When observing the reached accuracies as a function of the necessary time, it is obvious, that for our simulation higher side lengths lead to far longer computation times while not offering performance improvements after a side length of 14. It should ne noted, that for the physical system more parameters should not lead to an increased training time because all adjustments and measurements can happen in parallel.

Table 1: Training time and achieved testing accuracy for the different side lengths of the down-sampled MNIST images.

Side length	Training time in s	Testing accuracy
6	10	0.933
8	62	0.960
10	103	0.978
14	519	0.988
18	1398	0.978
22	3307	0.980

3.5 Batch size analysis

The influence of the batch size is evaluated in Table 2 (see also Figs. in appendix B.3). For this test a learning rate of 0.1, a patience of 3 epochs and maximum number of 10 epochs were chosen. The highest testing accuracy of 0.978 is achieved for a batch size of 1. At the same time, the computation time increases with lower batch sizes due to the increased number of gradient updating steps. Here, a similar behavior of the physical system would be expected. Overall, it is possible to already reach high testing accuracies after short training times using a larger batch size. For all batch sizes a similar training accuracy is achieved.

Table 2: Training time and achieved testing accuracy for the different batch sizes.

Batch size	Training time in s	Testing accuracy
1	2801	0.978
2	501	0.965
4	785	0.945
8	425	0.968
16	207	0.973
32	166	0.970
64	87	0.970
128	56	0.965
256	40	0.963
512	32	0.973

3.6 Loss function comparison

The influence on the performance of the loss function was investigated by comparing the mean square error loss function and the softmax cross-entropy loss function (see Fig. 18). For the loss functions a testing accuracy of 0.970 and 0.968 is reached respectively, therefore both seem to

perform similarly well. The batch loss shows the loss for each mini-batch within one epoch. The difference of the absolute loss between the functions is due to the difference between their "target value". The mean square error tries to minimize the intensity in all detectors but the target one to zero, while the softmax function does not. Thus, it is not possible to compare the functions based on their absolute loss.

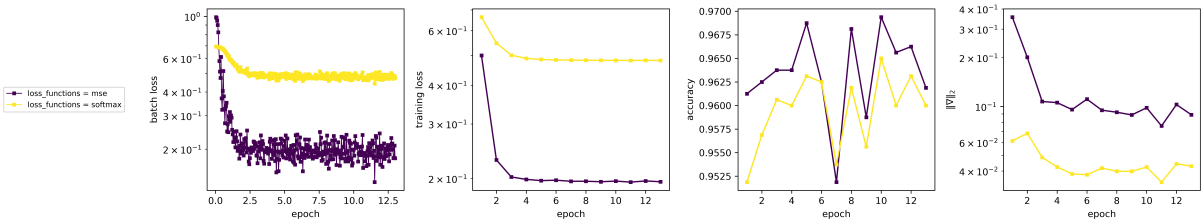


Figure 18: Batch loss, training loss, training accuracy and gradient for the two loss functions.

3.7 Hyperparameter theta_enc (in phase encoding)

The influence of the encoding hyperparameter `theta_enc` is examined for the phase encoding (see Tab. 3, see also Fig. in appendix B.4). It does not have to be considered for the amplitude encoding because it is a factor which would increase the value of all inputs in the same way. In phase encoding however, this parameter scales the phase according to $E_j = \exp\{(i \cdot \theta \cdot \pi \cdot V_j)\}$. By this we introduce another non-linearity that is relevant even with normalized energy. For low values of `theta_enc`, the training takes comparatively long, while reaching a low testing accuracy. The highest testing accuracy of 0.983 is reached for `theta_enc` of 0.750.

Table 3: Relevant results for different hyperparameters `theta_enc` for phase encoding.

Theta_enc	Epochs	Training accuracy	Testing accuracy	Training time in s
0.25	8	0.878	0.500	102
0.5	8	0.933	0.583	101
0.75	6	0.946	0.983	88
1.0	6	0.968	0.980	73
1.5	6	0.970	0.978	73
2.0	5	0.973	0.953	69

3.8 Our best model

We reached our highest testing accuracy of 0.990, meaning only 4 of the 400 test samples were misclassified. The used parameters are phase encoding with `theta_enc`=0.75, energy-normalization, $m = 14$, mean square error loss, a learning rate of 1.0, a batch size of 64 and Haar initialization. The training stopped after 5 epochs due to reaching the convergence criteria based on a patience of 1 and `min_delta`=0.0001. Switching from random to Haar initialization and from amplitude to phase encoding appears to offer significant improvements. Here, the training took 129 s, while the inference time is only 7.86 ms per sample.

3.9 Outlook

Further tests, that could be performed using our implemented model, would be a variation of the detector positions, a study of the optimal parameters for other mesh architectures as well as a resistance analysis studying the influence of losses in the beam splitters and fibers. While an expansion to more digits would be interesting, minor changes to some of the implemented functions would be necessary.

4 Discussion and critical analysis

4.1 Bias-variance trade-off analysis

To examine our optical system, a bias-variance trade-off analysis is performed considering the results of the training. The bias is related to underfitting and describes errors which are caused when a model is too simplistic, while the variance refers to overfitting and a too high complexity.

The use of a MZI mesh limits our model to unitary matrices. Additionally, the only non-linearity present in our model is the detection as intensity. This could be changed by adding a non-linearity between layers (ONU). Thermal and detector noise, imperfections in the fabrication of the chip and losses at the beam splitters and fibers might lead to additional errors contributing to the variance. Furthermore, the number of layers limits the number of adjustable parameters. Therefore, a low number of layers will contribute to the bias, while a high number of layers contributes to the variance. A similar argument can be made for the side length of the images. When too few pixels are used, relevant information is lost, while for too many pixels, the model trains on "noise". Therefore, the best accuracy can be reached when a reasonable trade-off between these aspects is found, as demonstrated in the previous section on our best-performing model.

4.2 Implementation challenges

The main challenge of the computational as well as physical implementation of this model is the estimation of the gradient. To calculate the gradient, the time-reversal interference method is used [21]. In the physical implementation, the non-measurability of the electric fields and the consequent need to use the measurable intensities for the computation lead to further complexity.

In our computational implementation the high runtime can pose a challenge. The main contribution appears to be the gradient update. It would be possible to reduce the runtime by exchanging the element-wise loop-based update for a matrix-based solution. Furthermore, the function `layer_matrices_separate` which constructs the matrices might be optimized by taking advantage of the sparse block-based structure and exchanging the appending to a list to the storage in a numpy array.

The chosen number of samples of 2000 seems to be rather low. It might make sense to repeat the tests with more samples to ensure that the dataset shows a realistic representation of the variance of the written digits.

4.3 Estimated energy consumption

The consumed energy of the physical model depends on the laser, the control of the encoding, the control of the phase shifters, the detectors and the computing for the encoding and decoding. Compared to the energy consumption of the laser and an assumed electro-optic control of the phase shifters, the other contributions are probably negligible. While the energy consumption of the electro-optic modulators should be constant over time, the energy consumption of the laser depends on the time that the laser is turned on, and therefore the time required for one processing of one sample times the number of processing steps within the training. After training of the model, for the testing only one forward propagation of the light is necessary.

5 Conclusion

We achieved the best classification result based on testing accuracy for the linear regression, where we calculated a testing accuracy of 0.998 for m=6 and energy normalization of the image vectors. For our model, we achieved the highest testing accuracy of 0.990 after 5 epochs for the use of phase encoding with `theta_enc`=0.75, energy-normalization, $m = 14$, mean square error loss, a learning rate of 1.0, a batch size of 64 and Haar initialization, a patience of 1 and `min_delta`=0.0001.

For the binary classification of digits "1" and "7", the linear regression outperformed our model, as this problem appears to be linearly separable. If the classification was expanded to include more digits or when two other digits were chosen, these results might change. Nevertheless, using the optical system it was possible to reach a testing accuracy close to the result of the linear regression.

As was shown, the model's performance can be improved by optimizing the hyperparameters, providing insights that might be transferred to the physical implementation. The Haar initialization outperformed the random initialization of parameters. The performance does not improve infinitely with increasing layer count and side length of the images, but it reaches an optimum for certain parameters. Before fabricating a physical chip, it is important to optimize these parameters.

In comparison to the physical implementation, in the simulation it was possible to "measure" the electrical field at each position, simplifying the method. In a physical system only the intensities would be accessible. Thus the training method becomes more complicated, as the "time-reversal interference method" should be applied. It would be possible to model this measurement protocol as well. The most time-consuming step of our model is the gradient update. But the training does not seem to scale with the number of parameters, enabling the simulation of large models. We would expect to observe the same behavior for the physical model.

As an outlook, it would be interesting to expand the model to more digits and to include losses and noise to evaluate the changes in performance.

References

[1] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain.", Psychological Review **65**, 386 (1958) 10.1037/h0042519.

[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors", Nature **323**, 533 (1986) 10.1038/323533a0.

[3] A. Vaswani et al., "Attention is All you Need", in Advances in Neural Information Processing Systems, Vol. 30 (2017).

[4] T. Brown et al., "Language Models are Few-Shot Learners", in Advances in Neural Information Processing Systems, Vol. 33 (2020), pp. 1877–1901.

[5] A. Radford et al., *Improving Language Understanding by Generative Pre-Training* (OpenAI, 2018).

[6] J. Kaplan et al., "Scaling Laws for Neural Language Models", (2020) 10.48550/arXiv.2001.08361, arXiv:2001.08361, pre-published.

[7] J. Hoffmann et al., "Training compute-optimal large language models", in Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22 (Nov. 28, 2022), pp. 30016–30030.

[8] H. Ritchie, *How much energy do data centers and artificial intelligence use?*, (July 20, 2026) https://ourworldindata.org/how-much-energy-do-data-centers-and-artificial-intelligence-use (visited on 07/24/2026).

[9] V. Samborska, *Scaling up: How increasing inputs has made artificial intelligence more capable*, (Jan. 20, 2025) https://ourworldindata.org/scaling-up-ai (visited on 07/22/2026).

[10] G. Cybenko, "Approximation by superpositions of a sigmoidal function", Mathematics of Control, Signals and Systems **2**, 303 (1989) 10.1007/BF02551274.

[11] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators", Neural Networks **2**, 359 (1989) 10.1016/0893-6080(89)90020-8.

[12] M. Reck et al., "Experimental realization of any discrete unitary operator", Physical Review Letters **73**, 58 (1994) 10.1103/PhysRevLett.73.58.

[13] W. R. Clements et al., "Optimal design for universal multiport interferometers", Optica **3**, 1460 (2016) 10.1364/OPTICA.3.001460.

[14] Y. Shen et al., "Deep Learning with Coherent Nanophotonic Circuits", Nature Photonics **11**, 441 (2017) 10.1038/nphoton.2017.93, arXiv:1610.02365.

[15] S. Pai et al., "Matrix Optimization on Universal Unitary Photonic Devices", Physical Review Applied **11**, 064044 (2019) 10.1103/PhysRevApplied.11.064044.

[16] M. Hermans et al., "Trainable hardware for dynamical computing using error backpropagation through physical media", Nature Communications **6**, 6729 (2015) 10.1038/ncomms7729.

[17] T. W. Hughes et al., "Adjoint Method and Inverse Design for Nonlinear Nanophotonic Devices", ACS Photonics **5**, 4781 (2018) 10.1021/acsphotonics.8b01522.

[18] M. B. Giles and N. A. Pierce, "An Introduction to the Adjoint Approach to Design", Flow, Turbulence and Combustion **65**, 393 (2000) 10.1023/A:1011430410075.

[19] G. Veronis, R. W. Dutton, and S. Fan, "Method for sensitivity analysis of photonic crystal devices", Optics Letters, Vol. 29, Issue 19, pp. 2288-2290 (2004) 10.1364/OL.29.002288.

[20] S. Molesky et al., "Inverse design in nanophotonics", Nature Photonics **12**, 659 (2018) 10.1038/s41566-018-0246-9.

[21] T. W. Hughes et al., "Training of photonic neural networks through in situ backpropagation and gradient measurement", Optica **5**, 864 (2018) 10.1364/OPTICA.5.000864.

[22] F. Morichetti et al., "Non-Invasive On-Chip Light Observation by Contactless Waveguide Conductivity Monitoring", IEEE Journal of Selected Topics in Quantum Electronics **20**, 292 (2014) 10.1109/JSTQE.2014.2300046.

Appendices

A Analytical Derivation of the Local Gradient

In the following we derive an expression for the gradient that is specific to our system. Since the loss function depends on the field only through the two detector scores, the chain rule gives

$$\frac{\mathrm{d}\mathcal{L}(s_1, s_7)}{\mathrm{d}p} = \frac{\partial \mathcal{L}}{\partial s_1}\frac{\mathrm{d}s_1}{\mathrm{d}p} + \frac{\partial \mathcal{L}}{\partial s_7}\frac{\mathrm{d}s_7}{\mathrm{d}p}. \tag{3}$$

The specific choice of loss function enters only through the two prefactors $\partial\mathcal{L}/\partial s_c$. Both are real-valued, since $\mathcal{L}$ is real by construction. Everything that follows is independent of that choice.

The remaining derivatives $\mathrm{d}s_c/\mathrm{d}p$ describe how the detected intensities respond to a change of p. We write the score of channel n as $s_n = E_{\mathrm{out},n}(p)\, E^*_{\mathrm{out},n}(p)$ and treat field and conjugate field as independent variables. This yields

$$\frac{\mathrm{d}s_n}{\mathrm{d}p} = \frac{\partial s_n}{\partial E_{\mathrm{out},n}}\frac{\mathrm{d}E_{\mathrm{out},n}}{\mathrm{d}p} + \frac{\partial s_n}{\partial E^*_{\mathrm{out},n}}\frac{\mathrm{d}E^*_{\mathrm{out},n}}{\mathrm{d}p} = E^*_{\mathrm{out},n}\frac{\mathrm{d}E_{\mathrm{out},n}}{\mathrm{d}p} + E_{\mathrm{out},n}\frac{\mathrm{d}E^*_{\mathrm{out},n}}{\mathrm{d}p}. \tag{4}$$

Because p is a real-valued phase, differentiation commutes with complex conjugation, $\mathrm{d}E^*/\mathrm{d}p = (\mathrm{d}E/\mathrm{d}p)^*$. The two terms in (4) are therefore complex conjugates of each other and their sum reduces to twice the real part of either one. Inserting this into (3) and moving the real prefactors inside the real part, we obtain

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}p} = 2\ \mathrm{Re}\left\{\frac{\partial \mathcal{L}}{\partial s_1}\, E^*_{\mathrm{out},d_1}\frac{\mathrm{d}E_{\mathrm{out},d_1}}{\mathrm{d}p} + \frac{\partial \mathcal{L}}{\partial s_7}\, E^*_{\mathrm{out},d_7}\frac{\mathrm{d}E_{\mathrm{out},d_7}}{\mathrm{d}p}\right\}. \tag{5}$$

We now define the error field $\mathbf{\Gamma} \in \mathbb{C}^N$. It quantifies the sensitivity of the loss to a perturbation of the complex field amplitude in each output channel,

$$\Gamma_n \equiv \frac{\partial \mathcal{L}}{\partial E^*_{\mathrm{out},n}} = \begin{cases} \dfrac{\partial \mathcal{L}}{\partial s_c}\, E_{\mathrm{out},n}, & n = d_c, \quad c \in \{1, 7\}, \\ 0, & \text{otherwise}, \end{cases} \tag{6}$$

so that (5) takes the compact form

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}p} = 2\ \mathrm{Re}\left\{\mathbf{\Gamma}^\dagger \frac{\mathrm{d}\mathbf{E}_{\mathrm{out}}}{\mathrm{d}p}\right\}. \tag{7}$$

The benefit of the AVM already becomes visible in (6). The error field does not depend on p, so the same $\mathbf{\Gamma}$ serves every tunable phase in the mesh.

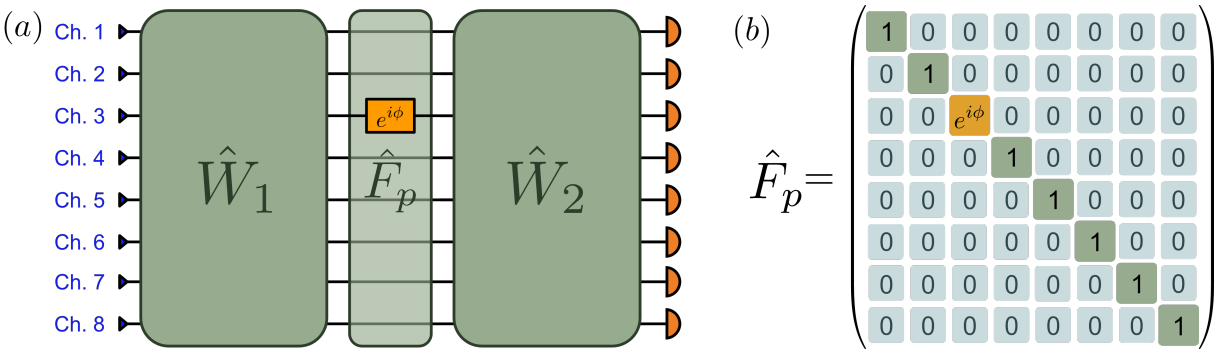


Figure 19: (a) Partition of the transfer matrix $\hat{W}$ of an eight-channel MZI mesh into a preceding mesh $\hat{W}_1$, an isolated layer $\hat{F}_p$ containing only the phase shifter p, and a subsequent mesh $\hat{W}_2$. (b) Transfer matrix of the isolated layer $\hat{F}_p$, here for a phase shift acting on channel $q = 3$.

It remains to evaluate the field derivative $\mathrm{d}\mathbf{E}_{\mathrm{out}}/\mathrm{d}p$. For this we isolate the position in the mesh at which p acts. Splitting off this artificial layer $\hat{F}_p$ leaves a preceding mesh $\hat{W}_1$ and a subsequent mesh $\hat{W}_2$, as shown in Figure 19,

$$\hat{W} = \hat{W}_2\,\hat{F}_p\,\hat{W}_1, \qquad \big(\hat{F}_p\big)_{kk} = \begin{cases} e^{ip}, & k = q, \\ 1, & k \neq q, \end{cases} \tag{8}$$

where q denotes the channel on which the phase shifter acts. Note that $\hat{F}_p$ is not a physical layer of the mesh. Each MZI contains two phase shifters, so the cut generally runs through a single MZI: its first beam splitter belongs to $\hat{W}_1$, while the remaining components belong to $\hat{W}_2$.

Differentiating (8) affects only the q-th diagonal entry,

$$\frac{\mathrm{d}\hat{F}_p}{\mathrm{d}p} = i\,e^{ip}\,\mathbf{e}_q\mathbf{e}_q^{\top}, \quad \text{with } \mathbf{e}_q \in \mathbb{R}^N, \quad \big(\mathbf{e}_q\big)_k = \delta_{qk},$$

Since $\hat{W}_1$ and $\hat{W}_2$ are independent of p, they act as constants. The unit vectors $\mathbf{e}_q$ form an outer product that projects onto the single channel q,

$$\frac{\mathrm{d}\mathbf{E}_{\mathrm{out}}}{\mathrm{d}p} = \hat{W}_2\,\frac{\mathrm{d}\hat{F}_p}{\mathrm{d}p}\,\hat{W}_1\mathbf{E}_0 = i\,e^{ip}\,\big(\hat{W}_2\mathbf{e}_q\big)\,\underbrace{\big(\mathbf{e}_q^{\top}\hat{W}_1\mathbf{E}_0\big)}_{\equiv\, A_p^{\mathrm{before}}}. \tag{9}$$

Grouping the terms this way isolates a scalar. The second bracket is the complex field amplitude A_p^{before} on channel q, immediately in front of the phase shifter. It is a quantity that the forward pass produces anyway, and on a physical chip its intensity is accessible through non-invasive on-chip monitors [22].

Inserting (9) into (7) gives a scalar product with $\mathbf{\Gamma}$. For a scalar, Hermitian conjugation reduces to complex conjugation, which lets us move $\hat{W}_2$ onto the error field,

$$\mathbf{\Gamma}^{\dagger}\hat{W}_2\mathbf{e}_q = \big(\mathbf{\Gamma}^*\big)^{\top}\hat{W}_2\mathbf{e}_q = \mathbf{e}_q^{\top}\hat{W}_2^{\top}\mathbf{\Gamma}^* = \big(\hat{W}_2^{\top}\mathbf{\Gamma}^*\big)_q.$$

The transpose $\hat{W}_2^{\top}$ describes propagation from the output ports back to the position of p. This identification rests on reciprocity of the lossless mesh. The conjugated error field $\mathbf{\Gamma}^*$ is the source injected at the detectors. We define its q-th component as the adjoint field amplitude,

$$A_p^{\mathrm{adj}} \equiv \big(\hat{W}_2^{\top}\mathbf{\Gamma}^*\big)_q.$$

Both A_p^{before} and A_p^{adj} are amplitudes taken in front of the phase shifter, each with respect to its own direction of propagation. Absorbing the phase factor into the forward amplitude, $A_p \equiv e^{ip}A_p^{\mathrm{before}}$, describes the field directly after the component. With $\mathrm{Re}\{i\,z\} = -\,\mathrm{Im}\{z\}$ we arrive at

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}p} = 2\,\mathrm{Re}\Big\{i\,A_p\,A_p^{\mathrm{adj}}\Big\} = -2\,\mathrm{Im}\Big\{A_p\,A_p^{\mathrm{adj}}\Big\} \tag{10}$$

which reproduces the result of [21], up to the factor of two absorbed into their definition of the error field.

Equation (10) is the result we set out to obtain. The gradient with respect to p follows from two field amplitudes at a single location, taken once for the forward field $\mathbf{E}_0$ and once for the backward-propagating error field. Both fields traverse the entire mesh in a single pass. The cost of a complete gradient evaluation is therefore independent of the number of trainable phases, which is what makes the approach attractive for large meshes.

On physical hardware one further step would be required. Photo detectors respond to intensity, so the complex amplitudes in (10) are not directly accessible. Hughes et al. address this with

the time-reversal interference method, which rests on three principles [21]. Reciprocity of the lossless mesh makes the backward pass a physically realizable propagation. Complex conjugation in the frequency domain corresponds to time reversal, which turns the counter-propagating adjoint field into a co-propagating one. Interfering it with the forward field then produces an intensity pattern whose cross term is the required overlap, isolated by subtracting the separately measured constant terms:

$$I_p = \left|A_p + (A_p^{\mathrm{adj}})^*\right|^2 = |A_p|^2 + |A_p^{\mathrm{adj}}|^2 + 2\ \mathrm{Re}\left\{A_p\, A_p^{\mathrm{adj}}\right\}.$$

The present work focuses on the training method rather than on a full simulation of the physical system. We therefore read the field amplitudes A_p and A_p^{adj} directly from the simulated forward and backward passes, which yields the same gradient (10) without modeling the measurement protocol.

B Additional figures

B.1 Learning rate analysis

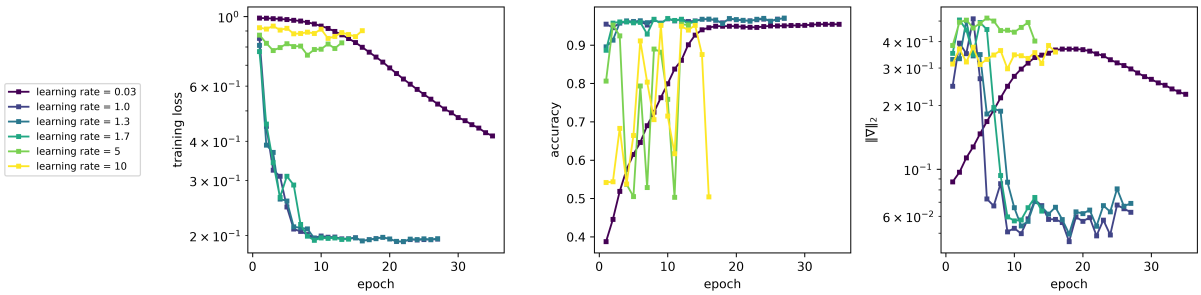


Figure 20: Training loss, training accuracy and normalized gradient for different learning rates for a batch size of 256.

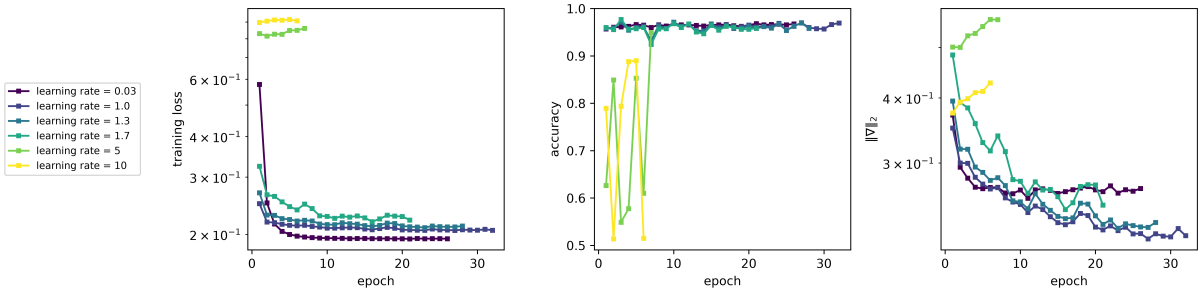


Figure 21: Training loss, training accuracy and normalized gradient for different learning rates for a batch size of 4.

B.2 Side length analysis (down-sampling)

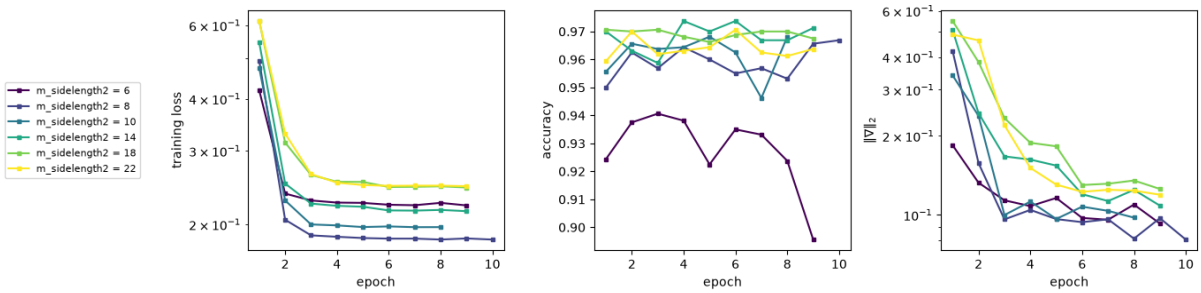


Figure 22: Training loss, training accuracy and normalized gradient plotted over the epochs for different side lengths of the down-sampled MNIST images.

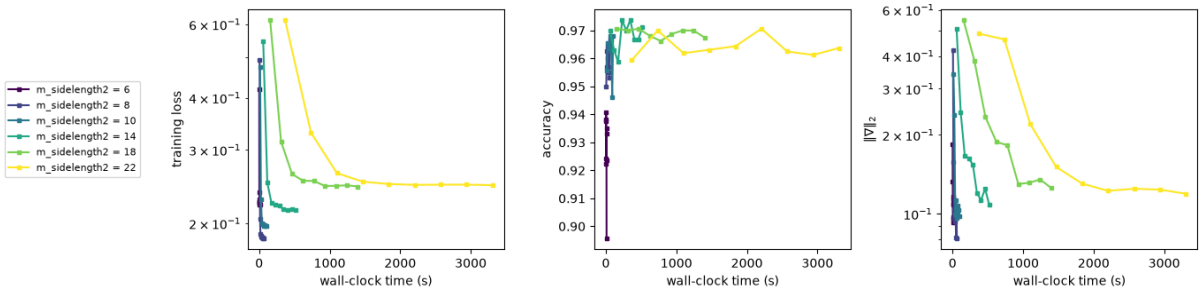


Figure 23: Training loss, training accuracy and normalized gradient plotted over the wall-clock time for different side lengths of the down-sampled MNIST images.

B.3 Batch size analysis

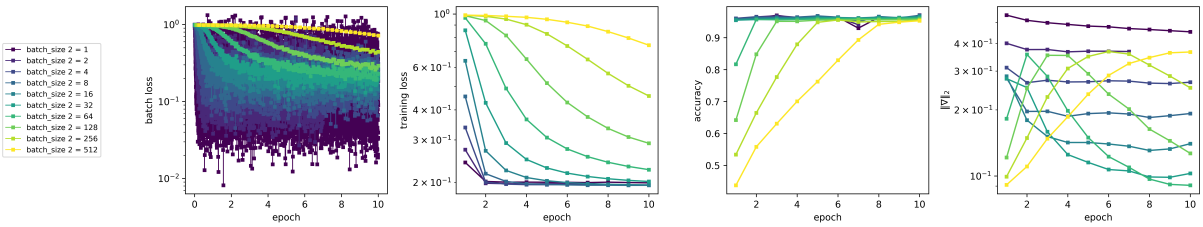


Figure 24: Batch loss, training loss, training accuracy and normalized gradient plotted over the epochs for different batch sizes.

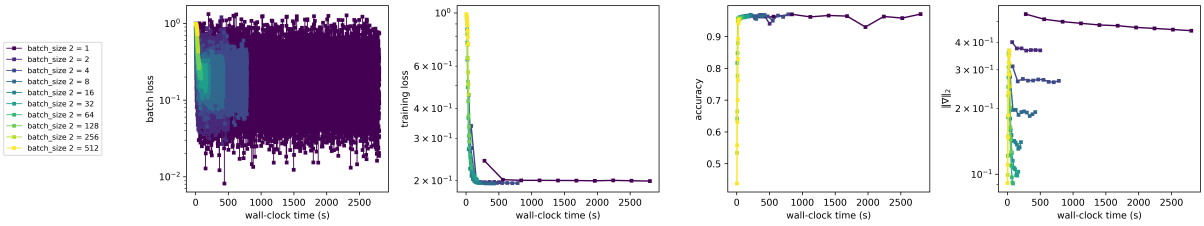


Figure 25: Batch loss, training loss, training accuracy and normalized gradient plotted over the wall-clock time for different batch sizes.

B.4 Hyperparameter theta_enc (in phase encoding)

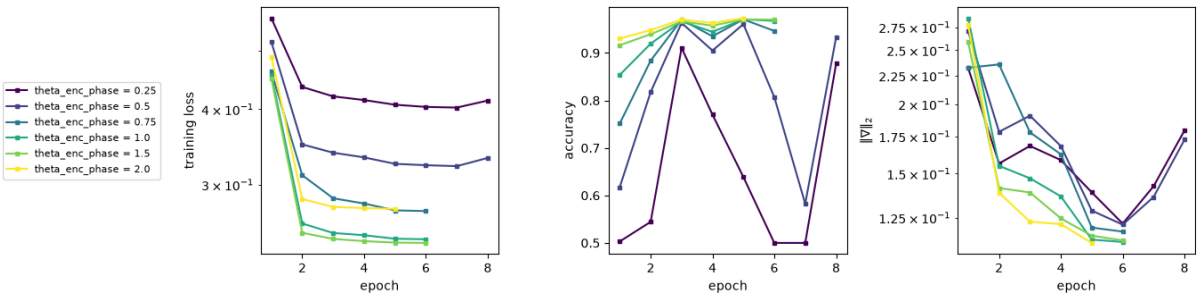


Figure 26: Training loss, training accuracy and normalized gradient plotted over the epochs for different values of `theta_enc` in phase encoding.

B.5 Architecture comparison

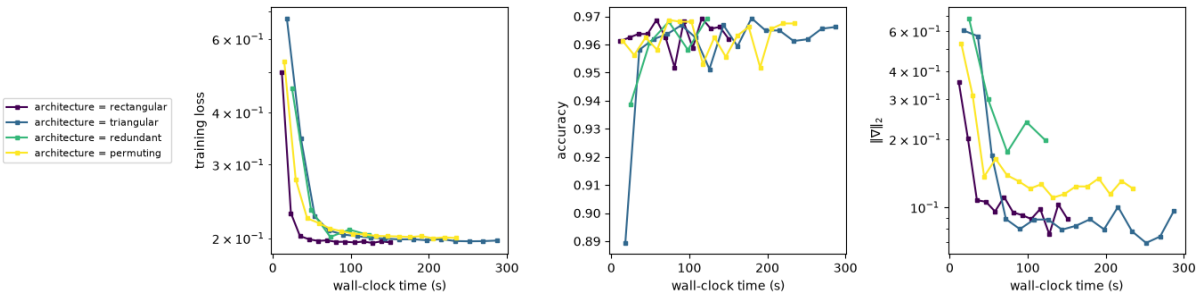


Figure 27: Training loss, training accuracy and normalized gradient for four different architectures of the MZI mesh.